Smart Agricultural Technology 9 (2024) 100547

Contents lists available at ScienceDirect

Smart Agricultural Technology

journal homepage: www.journals.elsevier.com/smart-agricultural-technology

Standalone edge AI-based solution for Tomato diseases detection

Yaqoob Majeed [a,b,1], Mike O. Ojo [a,1], Azlan Zahid [a,*]

[a] Department of Biological and Agricultural Engineering, Texas A&M AgriLife Research, Texas A&M University System, Dallas, TX 75252, USA
[b] Department of Electrical Engineering and Computer Science, University of Wyoming, Laramie, WY 82071, USA

ARTICLE INFO

Keywords:
CNN, Deep learning
Edge devices
Disease scouting
Plant disease
Edge-AI

ABSTRACT

Tomato yield is significantly affected by diseases, which are a continuous challenge for its production and pose threats to its global supply chain. Automatic and early detection of these diseases could help growers to swiftly adopt mitigation strategies to limit the disease spread, leading to improved production. Deep learning-based CNN approaches have been widely applied to detect tomato diseases. However, deep learning models are highly computationally demanding, resulting in a computational bottleneck for practical adaptation for agricultural applications such as disease detection and monitoring. Over the last few years, developments of open-source Edge systems have provided opportunities for low-cost and low-power consumption practical solutions for deep learning applications for agriculture. Therefore, the primary goal of this study was to evaluate the performance of standalone Edge-AI solutions for tomato leaf disease detection. To achieve this goal, firstly, this study employed lightweight deep learning networks to detect and differentiate tomato leaf diseases (bacterial spot, early blight, healthy, late blight, leaf mold, septoria leaf spot, two spotted spider mites, target spot, and yellow leaf curl virus). Then, these deep learning networks were deployed on low-cost and low-power consumption Edge devices to investigate their performance capabilities as standalone Edge-AI solutions for the early detection of tomato leaf diseases. Lightweight CNN based GoogleNet and MobileNetV2 deep learning networks achieved accuracies of up to 98.25 % compared to accuracies of 98.13 %, 98.13 %, 94.62 %, and 90.67 % of EfficientNetB0, ResNet-18, SqueezeNet, and NasNetMobile, respectively, in detecting tomato diseases. NVIDIA Jetson ORIN AGX and Nano significantly outperformed Raspberry Pi and Raspberry Pi with AI accelerator (Google Coral) in image classification, achieving classification times of 3.5 ms and 5.2 ms respectively, using SqueezeNet, compared to 15.3 ms and 10.5 ms on the Raspberry Pi devices. In addition, Raspberry Pi with Google Coral achieved the best cost/FPS performance of 0.14 compared to other Edge devices NVIDIA Jetson AGX Orin and NVIDIA Jetson Nano Orin with cost/FPS of 0.7 and 0.26, respectively. These results showed the potential of standalone Edge-AI solutions using low-cost and low-power consuming software and hardware resources for early tomato disease detections.

1. Introduction

Tomato is one of the important crops worldwide with production exceeding 189 million metric tonnes annually [1]. However, tomato diseases continuously challenge and threaten production. Accurate and timely identification of tomato diseases could help prevent disease spread, resulting in improved production and more stable market prices. By mitigating the effects of disease outbreaks, farmers can maintain consistent supply levels, which is crucial for accurate price forecasting [2,3]. Traditionally, tomato growers rely on visual observation of disease symptoms to identify and classify disease. However, these visual observation-based methods are dependent on human expertise level resulting in labor-intensive, time-consuming, and delayed information. Automatic and early detection of these diseases could help growers to swiftly adopt mitigation strategies to contain the spread of the disease, hence improving their tomato production. While computer vision excels at automatically recognizing and classifying diseases, it alone is not sufficient for real-time data applications in resource constrained environments like agriculture. This is because computer vision models typically require significant computational resources, which can be limited when deployed in the field. Edge-AI, on the other hand, processes data locally on devices like sensors or cameras, enabling real-time

---

* Corresponding author.
E-mail address: azlan.zahid@tamu.edu (A. Zahid).

[1] Authors contributed equally to the study

https://doi.org/10.1016/j.atech.2024.100547
Received 28 April 2024; Received in revised form 13 August 2024; Accepted 23 August 2024
Available online 30 August 2024
2772-3755/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

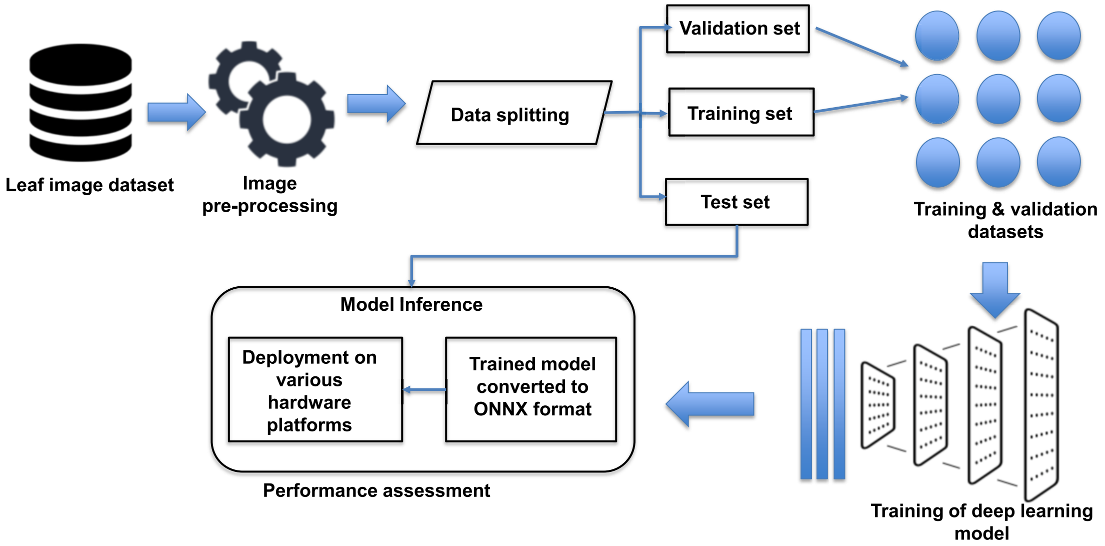

Leaf image dataset
Image pre-processing
Data splitting
Validation set
Training set
Test set
Training & validation datasets
Model Inference
Deployment on various hardware platforms
Trained model converted to ONNX format
Performance assessment
Training of deep learning model

Fig. 1. Overview of Edge-AI tomato leaf disease detection system showing the process from data processing to Edge-AI implementation.

analysis without the need for constant cloud communication. This approach not only reduces latency but also enhances the system's reliability and efficiency, making it well-suited for on-site disease detection. By combining Edge-AI with computer vision, the system can deliver faster, more reliable, and efficient disease detection directly at the source, ensuring timely interventions and minimizing crop losses. Therefore, the primary goal of this study is to evaluate the performance of standalone Edge-AI solutions for tomato leaf disease detection.

1.1. Related work

In recent years, computer vision-based methods are gaining popularity due to their effectiveness and efficiency in rapid and early disease detections [4,5]. Traditional computer vision-based techniques have been widely employed to detect plant diseases [6]. These computer vision techniques employed support vector machine (SVM) [7–9], combined Haar-like features with Viola-Jones algorithm [10], using the discrete wavelet transform and gray-level co-occurrence-based features for the SVM and K-Nearest Neighbor (K-NN) [11,12], combining local binary pattern (LBP) features with SVM [13], and combining Haralick texture, Hu moments and color features with SVM [14] and random forest [15] to detect and differentiate the tomato diseases. However, these traditional methods rely heavily on hand-crafted features of diseases, which hinders their success in practical disease identification. For the hand-crafted features, feature representation is limited, which inhibits the optimization and generalization of these approaches for disease identification.

To address this issue, deep learning based convolutional neural networks (CNN) offer higher hierarchical feature extraction and representation capabilities, resulting in increased disease detection accuracies and robustness [16]. Over the years, various researchers proposed deep learning-based CNN approaches for tomato disease detection and classification. For example, Thangaraj et al. [17] proposed multi-level feature fusion for CNN-based Xception network to detect tomato diseases. Bora et al. [18] employed multivariate-normal deep learning network to classify tomato diseases [18]. Similarly, modified InceptionResNetV2 was employed by Kaur et al. [19] to detect tomato leaf diseases. Nawaz et al. [20] employed the Faster R-CNN (faster region convolutional neural network) to detect and locate tomato diseases. Fuentes et al. [21] detected tomato diseases using the Faster R-CNN and SSD (single shot detector) networks. Wang et al. [22] used the ViT (vision transformer) based techniques to detect the tomato diseases. Ferentinos [23] used the transfer learning approach for different CNN-based deep learning networks, i.e. AlexNet, AlexNetOWTBn, GoogleNet, Overfeat, and VGG, to detect plant diseases. Roy et al. [24] proposed the PCA DeepNet by leveraging the CNN, PCA (principal component analysis), and GAN (generative adversarial network) to detect tomato leaf diseases. Similarly, Chen et al. [25] presented a tomato disease identification framework by combining the ABCK-BWTR (artificial bee colony-binary wavelet transform with retinex) and B-ARNet (both-channel residual attention network). However, deep learning models are highly computationally demanding, expanding in both width and depth with millions and billions of parameters, along with millions of floating-point operations (FLOPs). More extensive and complicated models offer increased accuracy but require significant processing resources [26,27]. This results in a computational bottleneck for the practical adaptation of deep learning models for agricultural applications such as disease detection and monitoring.

Over the last few years, developments of open-source Edge systems have provided opportunities for low-cost and low-power consumption practical solutions for deep learning applications in E-HealthCare [28], Smart Cities [29], and Smart Farms [30]. Advancements in hardware, including system-on-a-chip (SoC) accelerators, application-specific integrated circuits (ASICs), graphics processing units (GPUs), and central processing units (CPUs) helped to bridge the computational gap between the complex deep learning models and the resource constrained Edge systems. These Edge systems could provide the flexibility to execute inference procedures from the device itself with no required or limited internet accessibility, making it suitable for practical applications of deep learning in the agricultural field, such as tomato disease detections [31]. In the recent past, few studies have been reported for the implementation of deep learning networks on Edge devices for the Edge-AI (Edge-artificial intelligence) applications. For example, Anh and Duc [32] implemented the MobileNetV3 on Raspberry Pi 3 to monitor plant leaf diseases. Gonzalez-Huitron et al. [33] implemented and tested four CNN networks (MobileNetV2, NasNetMobile, Xception, and MobileNetV3) on Raspberry Pi 4 to detect tomato diseases. Similarly, Tarek et al. [34] implemented MobileNetV3 on Raspberry Pi 4 to detect tomato diseases. Moreira et al. [35] tested different CNN based algorithms (AlexNet, DenseNet, and SqueezeNet) for mobile applications to monitor plant diseases. Zhang and Chen [36] proposed the Edge cloud-based solution to detect tomato leaf curl disease. Similarly, Nag et al. [37] proposed a mobile application for tomato disease detection that leverages the DenseNet-121 network. However, these approaches either needed internet connectivity for data processing/ communication [35–37] or focuses solely on the comparison of different deep learning

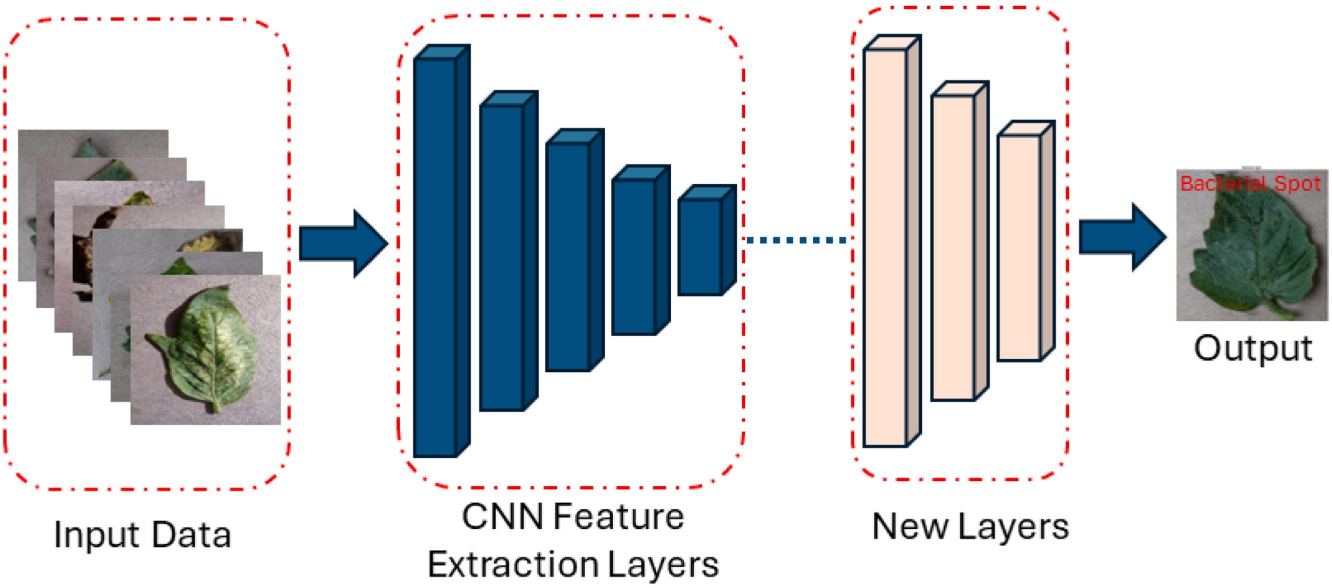


Fig. 2. Overall process of employing different CNN-based deep learning networks (extracting CNN features and addition of new layers) for tomato leaf diseases detection.

models on a single device (mostly Raspberry Pi) [32–34].

1.2. *Goals and objectives*

The overall goal of this study is to evaluate the performance of standalone Edge-AI solutions for tomato leaf disease detection. To achieve this goal, this study first employed lightweight deep learning networks to detect and differentiate tomato leaf diseases. Then, these deep learning networks were deployed on low-cost and low-power consumption Edge devices to investigate their performance capabilities as standalone Edge-AI solutions for the early detection of tomato leaf diseases. The results from this study would provide necessary performance indicators for the standalone Edge-AI solutions using low-cost and low-power consuming software and hardware resources for early tomato disease detections.

2. Materials and methods

Fig. 1 illustrates the system overview of our Edge-AI detection system, outlining the process for detecting tomato leaf disease on an Edge device. Subsequent subsections provide detailed descriptions of dataset preparation, light CNN architectures, deployments on Edge devices, and performance assessments.

2.1. *Dataset preparation*

As the primary goal of this study was to evaluate the performance of standalone Edge-AI solutions for tomato leaf disease detection, thus, tomato leaf diseases dataset was collected from PlantVillage dataset [38]. A tomato leaf dataset of 8568 images comprised of 9 different diseases (bacterial spot, early blight, healthy, late blight, leaf mold, septoria leaf spot, two spotted spider mites, target spot, and yellow leaf curl virus) were extracted from the PlantVillage dataset. Each image of the dataset has 256 × 256-pixel resolution. The dataset was randomly divided for training (∼80 % of total dataset: 6856 images), validation (∼10 % of total dataset: 857 images), and testing (∼10 % of total dataset: 855 images) datasets. To avoid overfitting and improve the generalizability of the deep learning networks, data augmentation was utilized to generate diverse training datasets. This study utilized random X reflection, random rotation, and random scaling to generate variation and augment the training dataset. For X reflection, each image was randomly reflected horizontally (left-right direction) with 50 % probability. For rotation, each image was rotated with a randomly picked angle between -90° and 90° While for scaling, each image was scaled with randomly picked scaling factors between 1 and 2. These augmentation techniques were applied randomly to each training image before feeding to the network.

2.2. *Lightweight CNN architectures*

In deep learning, convolution is used to extract the feature maps of desired objects in an image. The size of feature maps generated from convolutional layers depends on various parameters such as stride, depth, filter type, filter size, and padding. Eq. (1) shows the process of feature map generation using the convolutional layers:

$$F_j^n = b_j^n + \sum_k w_{jk}^n \star x_k^{n-1} \tag{1}$$

Where, F_j^n represents the feature map generated at n^{th} convolutional layer using the input from previous layer represented as x_k^{n-1} convolved with the weight matrix $\left(w_{jk}^n\right)$. While b_j^n represents the bias term to induce non-linearity helping to capture complex patterns in input.

Due to the higher-hierarchical feature capturing capabilities of CNNs, CNN-based deep learning networks were utilized to detect and identify tomato leaf diseases accurately and robustly. Additionally, as the objective was to deploy these deep learning networks for the Edge-AI applications, therefore, only CNN architectures which have less than 12 million parameters were considered. A total of six lightweight CNN architectures were implemented to detect tomato leaf diseases: ResNet-18 [39], EfficientNetB0 [40], NasNetMobile [41], SqueezeNet [42], MobileNetV2 [43], and GoogleNet [44] having 11.7 million, 5.3 million, 5.3 million, 1.24 million, 3.5 million, and 7 million parameters, respectively, which were trained on ImageNet dataset [45]. These networks are initially trained on large-scale datasets such as ImageNet for general image classification tasks. To adapt these networks for the detection of tomato leaf diseases, the input layers, weights, and final classification layers need to be replaced and fine-tuned to match the specific number of classes and nuances of the tomato leaf diseases. During fine-tuning, the overall architecture and complexity (number of parameters and layers) of the CNN models largely remain unchanged. However, fine-tuning enables the models to learn specific features relevant to the

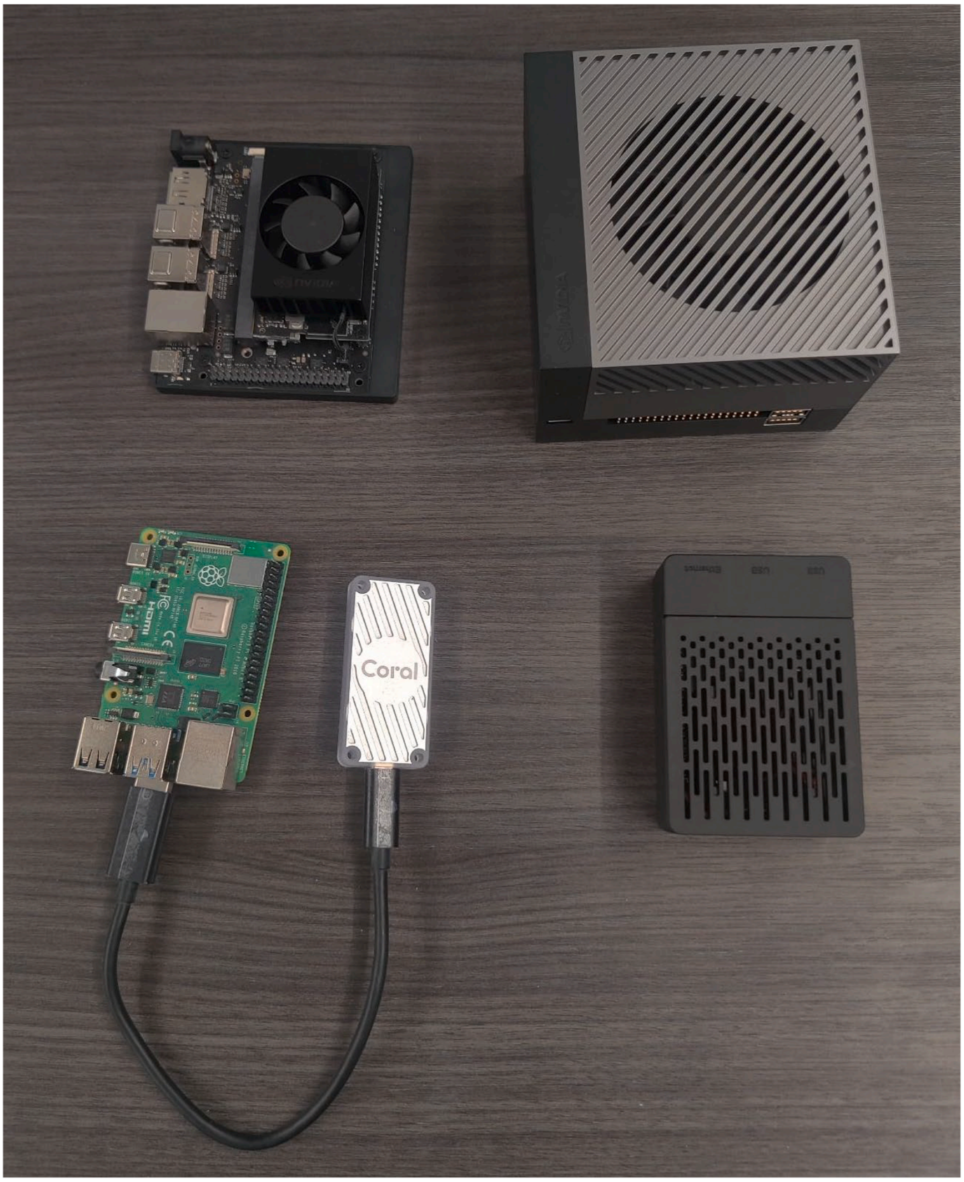


Fig. 3. The analyzed Edge devices used for the Edge-AI system. Top left: NVIDIA Jetson Orin Nano, Top right: NVIDIA Jetson AGX Orin, Bottom left: RPI 4 Model B with Google Coral, Bottom right: RPI 4 Model B.

custom tasks and improves their ability to handle vanishing gradients. This results in enhanced performance and reliability. The following paragraph discusses the architectural differences of each network and details the modifications made to specific layers for fine-tuning the networks for effective tomato leaf disease detection. Fig. 2 shows the overall process of employing CNN-based deep learning networks for tomato leaf diseases detection.

ResNet-18 consists of a residual block made up of skipped connections and 18 convolutional layers to extract the higher-level features. EfficientNet consists of MBConv (mobile-inverted bottleneck convolution) block to map narrow input and output layers to wider middle layers. EfficientNet comprises seven variants from B0 to B7, depending on their block sizes. In this study, EfficientNetB0 was employed because of its low latency and least number of parameters. NasNetMobile works on the NasNet search space process consisting of integrated convolutional layers and filter banks. Input images of the training dataset were normalized to 224 × 224 pixels to meet the input size requirements of the NasNetMobile. MobileNetV2 is based on depth-wise separable convolutional layers to apply point-wise and depth-wise convolutional operations to reduce the required computational power and model-complexity. GoogleNet utilizes the increased depth and width by convolutional layers and made decisions based on Hebbian principle and multi-scale processing to optimize the network performance with minimum required parameters. ResNet-18, EfficientNetB0, NasNetMobile, MobileNetV2, and GoogleNet requires the input layer size of 224 × 224 pixels, therefore, input images of training dataset were normalized to 224 × 224 pixels to meet the input size requirements of these networks. Also, the classification layers of these networks consisted of fully connected layers. Therefore, fully connected layers of these networks were replaced by a new fully connected layer with a weight learning factor and bias learn factor of 10 each. Then, probabilities of each class belonging to a specific class were computed by replacing the existing classification layer with a new classification layer of 9 classes. While, SqueezeNet consists of Fire modules, squeeze convolutional layers, and late down-sampling principle to achieve the comparable accuracy with least number of parameters. Input images of training dataset were normalized to 227 × 227 pixels to meet the input size requirements of the SqueezeNet. The classification layers of SqueezeNet comprised of

Table 1
The primary specifications of the Edge platforms explored in this study are provided along with their respective commercial prices at the time of publication.

Features	NVIDIA Jetson Orin Nano	NVIDIA Jetson Orin AGX	Google Coral	Raspberry Pi 4 Model B (RPI 4)
Size	100 mm x 79 mmx 21 mm	110 mm x 110 mm x 71.65 mm	65 mm x 30 mm	95 × 70 × 24.9 mm
CPU	6-core Arm® Cortex®-A78AE v8.2 64-bit CPU 1.5 MB L2 + 4 MB L3	12-core Arm® Cortex®-A78AE v8.2 64-bit CPU 3 MB L2 + 6 MB L3	NXP i.MX 8 M SoC (quad Cortex-A53, Cortex-M4F)	Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC
HW Accelerator	1024-core NVIDIA Ampere architecture GPU with 32 Tensor Cores	2048-core NVIDIA Ampere architecture GPU with 64 Tensor Cores	Integrated GC7000 Lite Graphics	N/A
Memory	8 GB 128-bit LPDDR5 68 GB/s	64 GB 256-bit LPDDR5 204.8 GB/s	4 GB LPDDR4	8 GB LPDDR4
Power	7 W - 15 W	15 W - 60 W	15 W	15 W
Weight	173 g	880 g	19.6 g	45.5 g
Peak Performance	40 TOPS	275 TOPS	4 TOPS	N/A
Price	$499	$1999	$60	$75

convolution 2D layers instead of fully connected layers. Therefore, a new convolution 2D layer with weight learning factor and bias learn factor of 10 each was defined and replaced with existing layer to classify the diseases. Then, probabilities of each class belonging to specific class were computed by replacing existing classification layer with new classification layer of 9 classes.

To optimize the convergence of these networks, SGD (stochastic gradient descent) was utilized, represented in Eq. (2), while cross-entropy loss was utilized as an objective function, represented in Eq. (3):

$$\theta_{t+1} = \theta_t - \eta\nabla_\theta L(\theta_t; x_i, y_i) \tag{2}$$

$$\text{Loss} = -\sum\nolimits_{k}^{i=1} y_i \log(p_i) \tag{3}$$

Where, θ_t is the parameters at iteration t, η shows the learning rate, and $\eta\nabla_\theta L(\theta_t; x_i, y_i)$ is the gradient of the loss function L w.r.t θ evaluated at (x_i, y_i). Whereas in equation x, k is the number of classes, y_i is the binary indicator, and p_i is the predicted probability for class i.

To make the network converge faster while keeping loss minimum, the initial learn rate and minimum batch size were kept at 0.0001 and 10 (after several trial-and-error runs), respectively. Also, the sequence of training data was shuffled randomly to avoid overfitting the network. Training of these networks were carried out on a workstation equipped with an NVIDIA® RTX™ A6000 GPU (48 GB) paired with an Intel® Xeon® Gold 6246R Processor, along with 256 GB of RAM. The RTX A6000 boasts cutting-edge RT Cores, Tensor Cores, and CUDA cores, enabling outstanding performance in rendering, AI, graphics, and computing. Its memory interface is 384-bit, with a memory bandwidth of 768 GB/s. This GPU delivers impressive computational power, capable of achieving peak performances of 38.7 Tera floating-point operations per second (TFLOPS), with 75.6 TFLOPS for single-precision, and 309.7 TFLOPS for RT Core and Tensor operations, respectively.

2.3. *CNN models deployments on edge devices*

Leading brands in the industry that are making significant contributions to the advancement of AI at the edge are NVIDIA, Qualcomm, and Google. For example, Google Coral [46] from Google offers a cost-effective solution for deploying complex algorithms. In [47], the authors utilize various CNN model and deployed it using Google Coral for person reidentification (re-ID). On the other hand, NVIDIA [48], the top brand in the industry that has made a significant contribution to the advancement of Edge-AI, comes in a variety of forms. NVIDIA Jetson variants, like Nano Orin and AGX Orin, are affordable, have a small form factor, are lightweight, and can effectively execute complex algorithms using several CNN layers. An additional significance of NVIDIA Jetson variations is the implementation of CPU-GPU architecture [49], in which the CPU initiates the firmware and the GPU with CUDA capability has the ability to expedite intricate machine-learning tasks.

In our study, we evaluated several Edge computing devices including the Raspberry Pi 4 Model B (RPI 4) with or without Google Coral, NVIDIA Jetson Orin Nano, and NVIDIA Jetson AGX Orin as illustrated in Fig. 3. Table 1 provides a comparison of the key specifications of these selected hardware options.

Google Coral serves as a hardware USB accelerator, augmenting the system with an Edge tensor processing unit (TPU) coprocessor. Featuring a USB-C socket, it connects to a host computer to facilitate accelerated deep learning inferencing. The on-board Edge TPU, an application-specific integrated circuit (ASIC) developed by Google, efficiently accelerates TensorLite models while consuming minimal power, achieving a peak performance of 4 trillion operations per second (4 TOPS) using only 2 W of power. Leveraging Google Coral for ML processing reduces latency and enhances data privacy.

NVIDIA offers a range of boards featuring embedded computers equipped with dedicated GPUs for hardware acceleration. Among the NVIDIA Edge devices examined in this study are the NVIDIA Jetson Orin Nano and NVIDIA Jetson Orin AGX.

NVIDIA Jetson Orin Nano, introduced in March 2023, boasts a compact design with abundant connectors and can deliver performance of up to 40 TOPS, catering efficiently to various AI applications. Equipped with an Ampere GPU and a 6-core ARM CPU, the Jetson Orin Nano facilitates multiple concurrent AI application pipelines and high-performance inference. Its modules offer up to 80 times the performance of the Jetson Nano, enabling it to handle modern AI models, including transformers and advanced robotics models.

NVIDIA Jetson Orin AGX, launched in May 2022, offers up to 275 TOPS of AI performance, with power configurable between 15 W and 60 W. It excels in deploying large and complex modules for tasks such as natural language understanding, 3D perception, and multi-sensor fusion. With up to 8 times the performance of the Jetson AGX Xavier in a compact form factor, the Jetson Orin AGX is ideal for developing a variety of AI applications. It integrates the new NVIDIA Ampere architecture featuring 2048 NVIDIA CUDA cores and 64 tensor cores, along with a 12-core Arm Cortex-A78AE v8.2 64-bit CPU with 3 MB L2+ 6 MB L3 cache. Supporting multiple concurrent AI inference pipelines, it includes onboard 64GB eMMC, 204 GB/s of memory bandwidth, and 64GB of DRAM. It also offers high-speed interface support for multiple sensors including 22 lanes of PCIe Gen4, 1 × 10GbE, Display Port, 16 lanes of MIPI CSI-2, USB 3.2 interfaces, and a 40-pin header.

RPI 4 on the other hand, is one of the most used embedded platforms with a large variety of usage. This cost-effective, compact computer board integrates onboard electronics that support an extensive range of input and output peripherals. Powered by a 64-bit Broadcom BCM2711 quad-core Cortex-A72 SoC running at 1.8 GHz, it offers options for 1/2/4/8 GB of RAM. The board is equipped with a Gigabit Ethernet interface, dual-band 2.4 GHz and 5 GHz IEEE 802.11b/g/n/ac WiFi, two USB 2.0 ports, two USB 3.0 ports, two micro-HDMI connectors, and a microSD card slot. Notably, it lacks dedicated GPU capabilities.

Following the training of various models, we delve into the

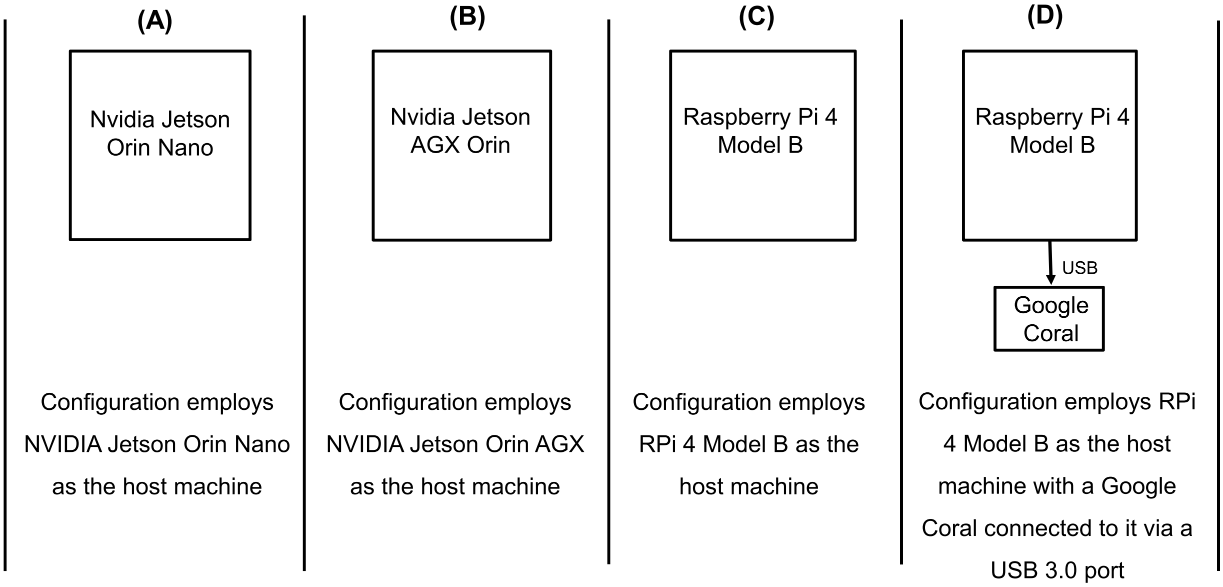

(A)
Nvidia Jetson Orin Nano
Configuration employs NVIDIA Jetson Orin Nano as the host machine
(B)
Nvidia Jetson AGX Orin
Configuration employs NVIDIA Jetson Orin AGX as the host machine
(C)
Raspberry Pi 4 Model B
Configuration employs RPi 4 Model B as the host machine
(D)
Raspberry Pi 4 Model B
USB
Google Coral
Configuration employs RPi 4 Model B as the host machine with a Google Coral connected to it via a USB 3.0 port

Fig. 4. Classification system schematic of Edge-AI system using (a) NVIDIA Jetson Orin Nano (b) NVIDIA Jetson Orin AGX (c) RPI 4 Model B (d) Raspberry Pi 4 Model B with Google Coral.

deployment phase onto hardware platforms. Within our experimental setup, we execute six distinct models: ResNet-18, EfficientNetB0, NasNetMobile, SqueezeNet, MobileNetv2, and GoogleNet, across four different configurations.

The first configuration employs a NVIDIA Jetson Orin Nano as the host machine, with specifications outlined in Table 1. The second configuration utilizes a NVIDIA Jetson Orin AGX, while the third configuration employs an RPI 4 Model B. In the fourth configuration, an RPI 4 Model B serves as the host machine, with a Google Coral connected to it via a USB 3.0 port. Corresponding diagrams for each configuration are depicted in Fig. 4.

To avoid platform limitations, we refrained from converting the models to TensorRT, which is compatible only with NVIDIA devices. Our comparison of various platforms necessitated an approach that would be inclusive of Raspberry Pi devices, where TensorRT cannot be utilized. TensorRT typically serves as a supplementary tool for training frameworks, leveraging GPU acceleration for swift network inference.

Instead, we opted for the Open Neural Network Exchange (ONNX) format, which offers compatibility across all platforms, facilitating robust comparison. ONNX serves as an intermediary framework bridging PyTorch models and TensorRT models. Since the model was originally trained using the MatLab and saved via as ".mat" file, conversion to the ONNX format was straightforward.

Furthermore, our setup across all devices utilized Docker containers, ensuring consistency. Each Docker image encapsulated all necessary libraries and packages, streamlining the deployment process.

2.4. Performance evaluation

The performance of the trained CNN architectures were evaluated using accuracy, precision, recall, and F1-score. Accuracy was defined as the ratio of correctly identified tomato diseases to the total number of samples (Eq. (4)). Precision was defined to check the actual number of positive classes from the given probabilities of positive classes (Eq. (5)). Sensitivity or recall was used to measure the predicted positive classes against the actual classes (Eq. (6)). While F1-score was used to evaluate the model accuracy as a harmonic mean of both precision and recall (Eq. (7)).

$$Accuracy = \frac{True\ Positive + True\ Negative}{True\ Positive + True\ Negative + False\ Positive + False\ Negative} \tag{4}$$

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive} \tag{5}$$

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative} \tag{6}$$

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{7}$$

The evaluation metrics considered for comparing and analyzing the inference performance across four Edge devices configurations include:

- CPU Utilization: This metric measures the rate of CPU usage during inference, expressed as a percentage based on the average usage of the available CPU cores. For instance, on the NVIDIA Jetson Orin AGX with 12 cores, the utilization of all 12 cores is averaged. The value ranges between 0 and 100.
- GPU Utilization: This metric measures the rate of GPU usage during inference, also expressed as a percentage ranging from 0 to 100.
- Average Power Consumption: This metric indicates the average power consumption of the devices utilized for inference, measured in watts (W)
- Classification Time: This metric denotes the time taken to classify a single image.
- Frames per Second (FPS): This metric quantifies the number of frames processed per unit of time during the classifier's execution. It is the reciprocal of the classification time.

3. Results and discussions

3.1. Tomato leaf diseases detection with lightweight CNN architectures

Training and validation loss and accuracies for all networks converges at the end, indicating the networks convergence capabilities without overfitting the training dataset. This helps to accurately identify the tomato diseases on the test dataset which was unknown to the network.

Table 2
Performance measures (Accuracy, Precision, Recall, and F1-Score) of CNN networks utilized for detecting tomato leaf diseases.

Model	Accuracy	Precision	Recall	F1-Score
GoogleNet	0.9825	0.9825	0.9829	0.9825
MobileNetV2	0.9825	0.9825	0.9827	0.9825
SqueezeNet	0.9462	0.9462	0.9477	0.9463
NasNetMobile	0.9076	0.9076	0.9347	0.9119
EfficientNetB0	0.9813	0.9813	0.9817	0.9812
ResNet-18	0.9813	0.9813	0.9816	0.9812

Table 2 summarizes the average accuracy, precision, recall, and F1-score of trained CNN networks on the test dataset. GoogleNet and MobileNetV2 helped to achieve the best performance. The highest average accuracy, precision, recall, and F1-score of 0.9825, 0.9825, 0.9829, and 0.9825, respectively, was achieved with GoogleNet and 0.9825, 0.9825, 0.9827, and 0.9825, respectively was achieved MobileNetV2 compared to other networks. The performance of ResNet-18 and EfficientNetB0 was almost the same but have lower accuracy, precision, recall, and F1-score compared to GoogleNet and MobileNetV2. NasNetMobile has the lowest performance on all the indicators compared to other networks. High recall score (0.9347) of NasNetMobile compared to accuracy of 0.9076, indicates that it suffers from a higher false positive rate. Fig. 5 shows the accuracy, precision, recall, and F1-score of each tomato leaf disease category. It can be observed that NasNetMobile struggled significantly with target spot and two-spotted spider mite diseases where recall is noticeably lower, suggesting a higher rate of false negatives. GoogleNet generally excelled across all classes, showing robust and reliable performance across all diseases. It is worth noting that the ResNet-18 network has the highest number of parameters (11.7 million) while its accuracy was lower than the GoogleNet (with 7 million parameters) and MobileNetV2 (3.5 million parameters). Additionally, the average accuracies of GoogleNet and MobileNetV2 were same while MobileNetV2 has half the parameters as of GoogleNet. With the least number of parameters (1.24 million), SqueezeNet has an accuracy of 0.9462 which could be used for scenarios where computational efficiency is a higher priority. As the number of training, validation, and testing images were the same for all networks and no overfitting evidence was found during training, it could be assumed that no positive correlation exists between the number of parameters and the accuracy of the network which could be an important factor while choosing the network for the Edge devices (further analysis on it is elaborated and discussed in detail in Section 3.2).

The ROC (receiver operator characteristic) curves were used to evaluate sensitivity (true positive rate) against the specificity (false positive rate) of each network across tomato diseases, as show in Figs. 6a-f. It could be observed that the operating points of every disease for all models were above the diagonal line (black dashed line), which means model performances for every disease were satisfactory (higher true positive rates compared to false positive rates). The values of area under curve (AUC) of each disease for all models were ≥0.99 (closer to 1) indicating the overall satisfactory performance. Additionally, as the curve shapes of each model for every disease bend towards the upper left corner which means satisfactory model performance for varying classification thresholds. For the NasNetMobile network, as shown in Fig. 6c, having the lowest average accuracy compared to other networks, the performance was mainly affected by target spot and two spotted mite diseases whose operating points were lower than 0.8 true positive rate. It means model incorrectly classified the diseases from/into these disease categories. Confusion matrices helped to sneak further into model performances to figure out the main reasons for the wrong classification of these diseases, as shown in Figs. 7a-f.

From the confusion matrix of NasNetMobile (Fig. 7c), it is evident that the target spot disease was the main reason for the lower performance of the network. The target spot was misclassified into two spotted mite disease (25 wrongly predicted instances) and healthy (20 wrongly classified predicted instances) classes. This could be due to the reason that NasNetMobile was not able to learn the hierarchical features of this

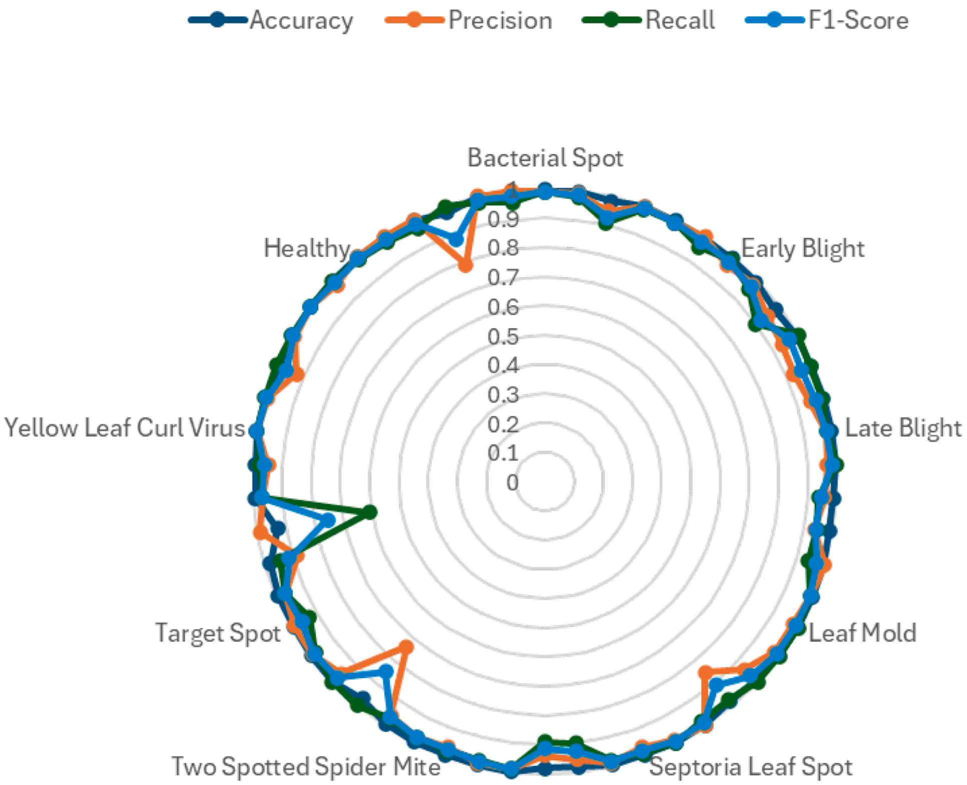
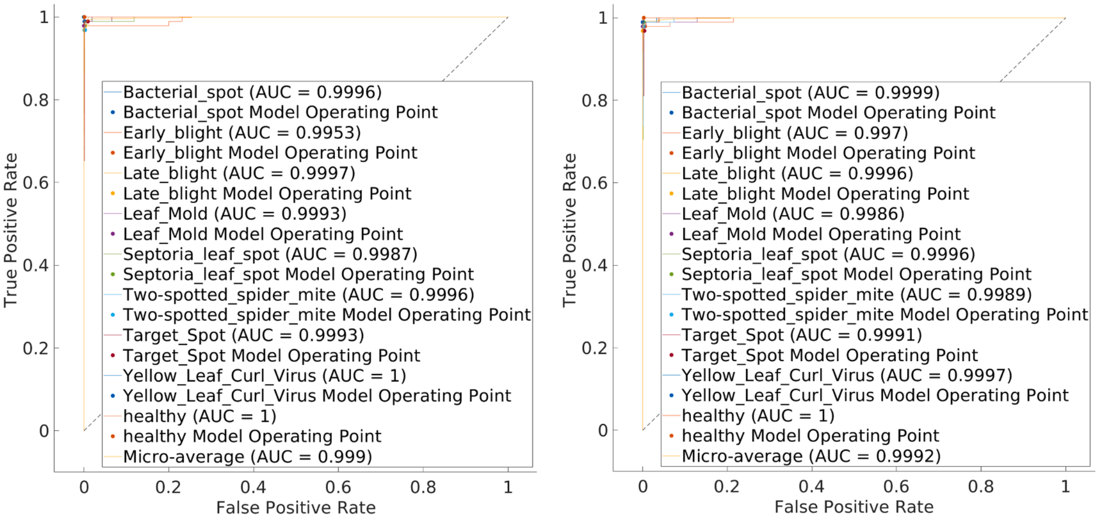
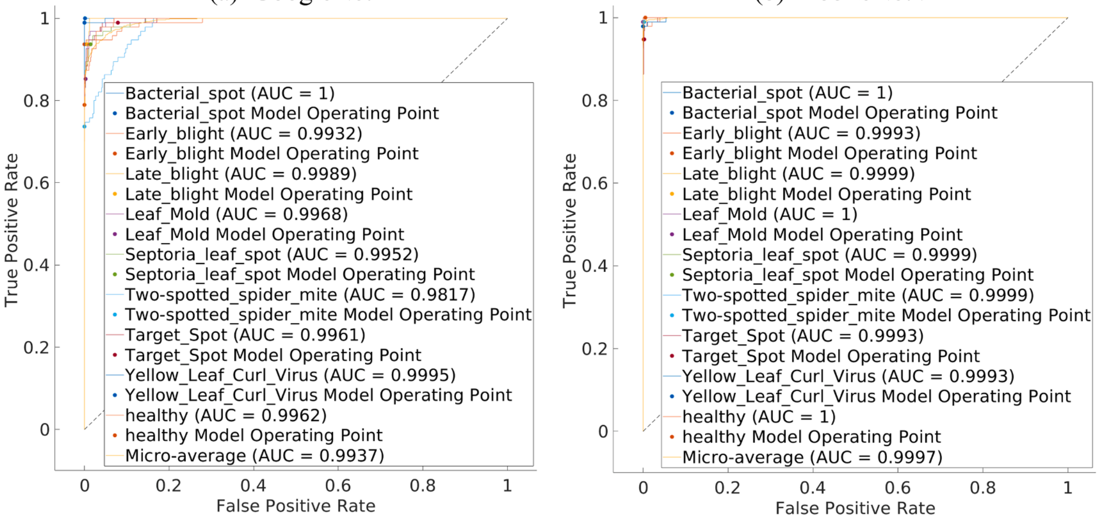


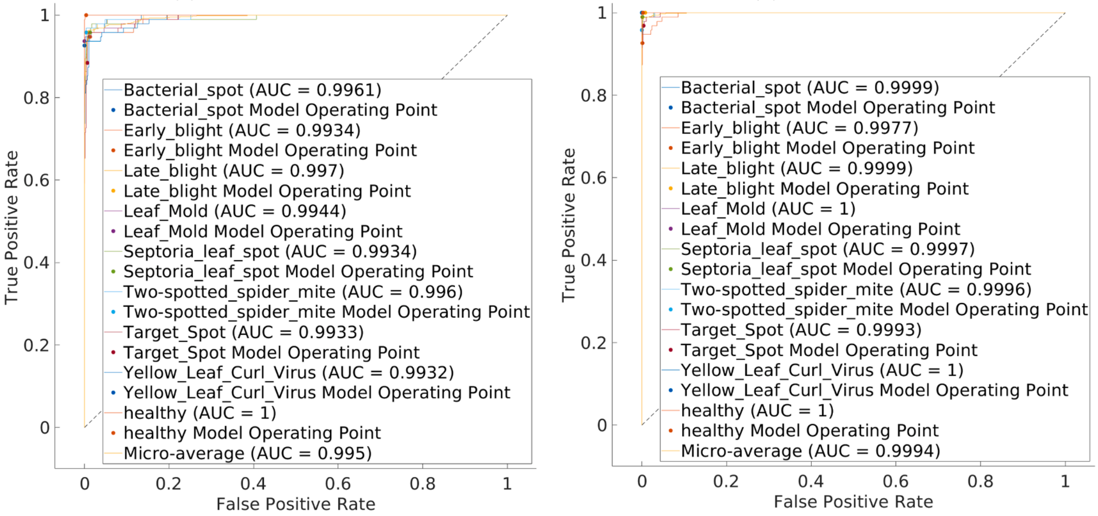
Fig. 5. Accuracy, precision, recall and F1-score of each tomato leaf disease category. CNN networks can be read as clockwise in the order of GoogleNet, MobileNetV2, SqueezeNet, NasNetMobile, EfficientNetB0, and ResNet-18.



(a) GoogleNet (b) MobileNetV2

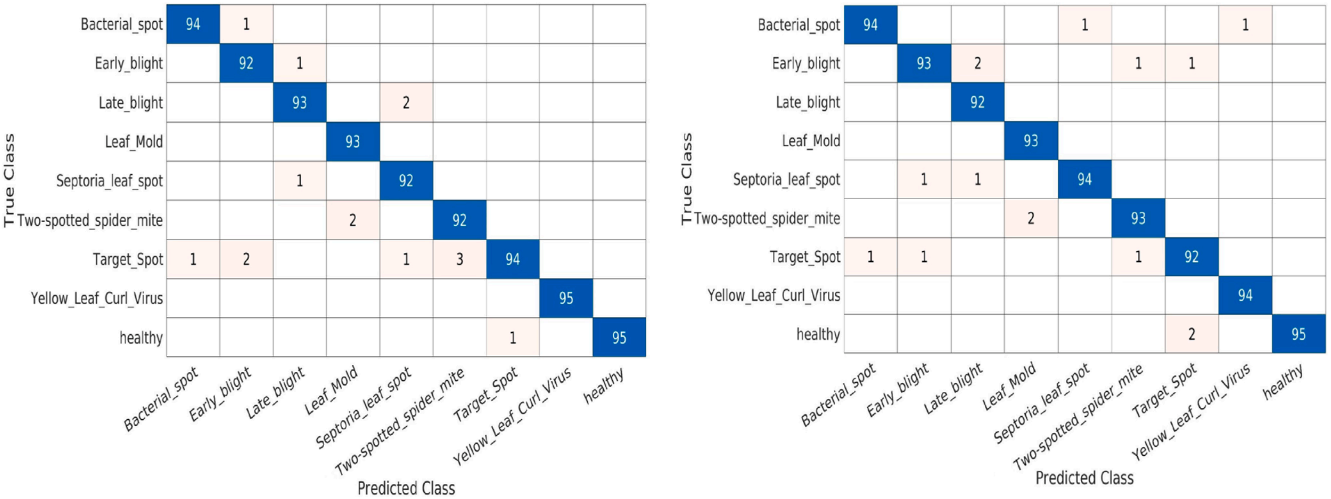


(c) NasNetMobile (d) ResNet-18



(e) SqueezeNet (f) EfficientNetB0

Fig. 6. ROC (receiver operator characteristic) curves representing the sensitivity (true positive rate) against the specificity (false positive rate) of GoogleNet (a), MobileNetV2 (b), NasNetMobile (c), ResNet-18 (d), SqueezeNet (e), and EfficientNetB0 (f).

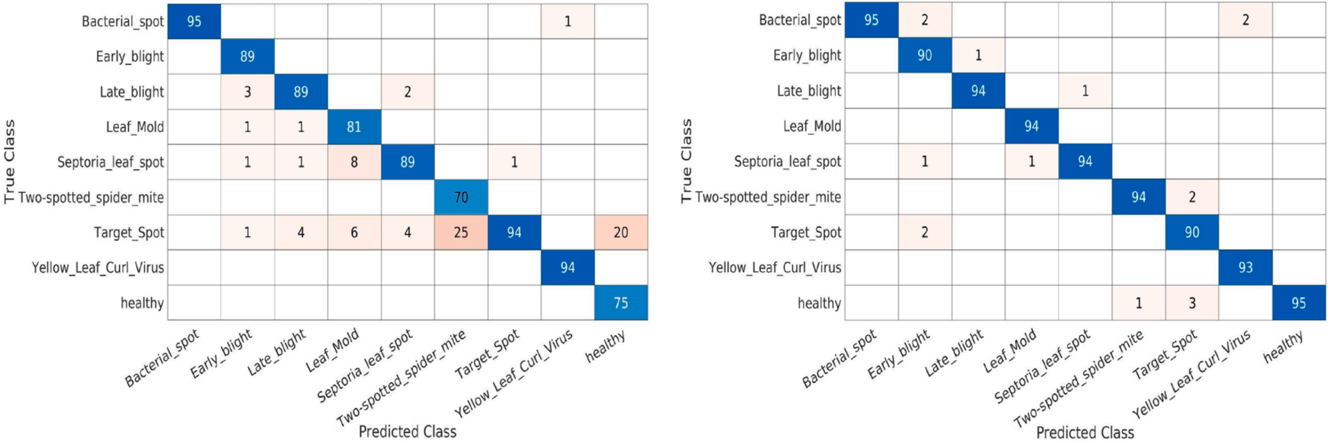


	Bacterial_spot	Early_blight	Late_blight	Leaf_Mold	Septoria_leaf_spot	Two-spotted_spider_mite	Target_Spot	Yellow_Leaf_Curl_Virus	healthy
Bacterial_spot	94	1							
Early_blight		92	1						
Late_blight			93		2				
Leaf_Mold				93					
Septoria_leaf_spot			1		92				
Two-spotted_spider_mite				2		92			
Target_Spot	1	2			1	3	94		
Yellow_Leaf_Curl_Virus								95	
healthy							1		95

(a) GoogleNet

	Bacterial_spot	Early_blight	Late_blight	Leaf_Mold	Septoria_leaf_spot	Two-spotted_spider_mite	Target_Spot	Yellow_Leaf_Curl_Virus	healthy
Bacterial_spot	94				1			1	
Early_blight		93	2			1	1		
Late_blight			92						
Leaf_Mold				93					
Septoria_leaf_spot		1	1		94				
Two-spotted_spider_mite				2		93			
Target_Spot	1	1				1	92		
Yellow_Leaf_Curl_Virus								94	
healthy							2		95

(b) MobileNetV2

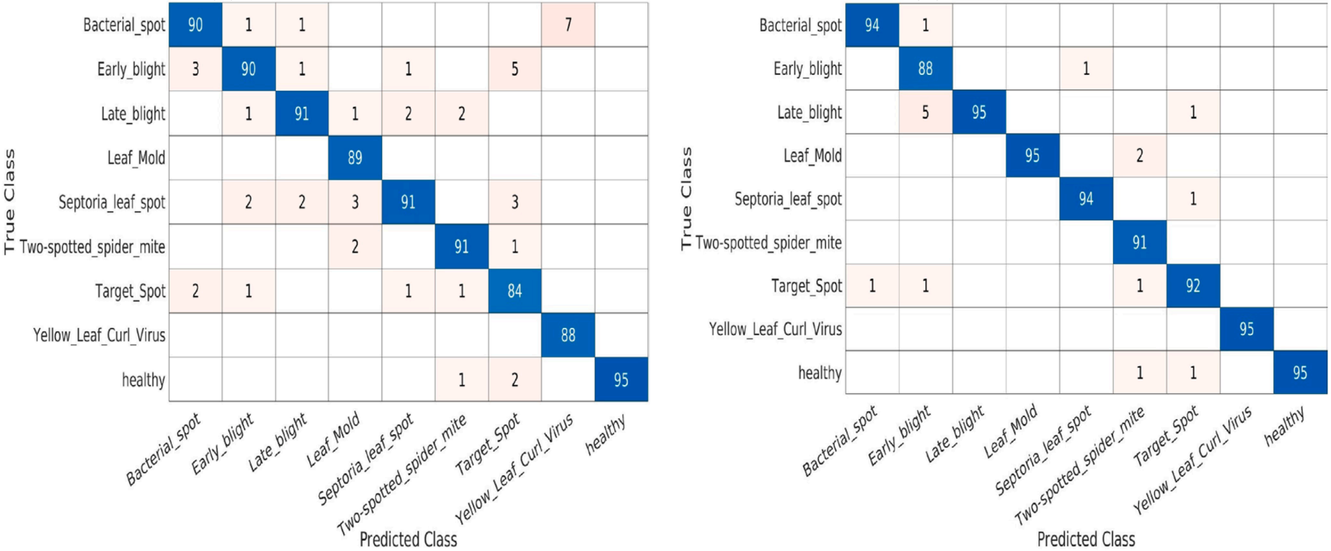


	Bacterial_spot	Early_blight	Late_blight	Leaf_Mold	Septoria_leaf_spot	Two-spotted_spider_mite	Target_Spot	Yellow_Leaf_Curl_Virus	healthy
Bacterial_spot	95							1	
Early_blight		89							
Late_blight		3	89		2				
Leaf_Mold		1	1	81					
Septoria_leaf_spot		1	1	8	89		1		
Two-spotted_spider_mite						70			
Target_Spot		1	4	6	4	25	94		20
Yellow_Leaf_Curl_Virus								94	
healthy									75

(c) NasNetMobile

	Bacterial_spot	Early_blight	Late_blight	Leaf_Mold	Septoria_leaf_spot	Two-spotted_spider_mite	Target_Spot	Yellow_Leaf_Curl_Virus	healthy
Bacterial_spot	95	2						2	
Early_blight		90	1						
Late_blight			94		1				
Leaf_Mold				94					
Septoria_leaf_spot		1		1	94				
Two-spotted_spider_mite						94	2		
Target_Spot		2					90		
Yellow_Leaf_Curl_Virus								93	
healthy						1	3		95

(d) ResNet-18



	Bacterial_spot	Early_blight	Late_blight	Leaf_Mold	Septoria_leaf_spot	Two-spotted_spider_mite	Target_Spot	Yellow_Leaf_Curl_Virus	healthy
Bacterial_spot	90	1	1					7	
Early_blight	3	90	1		1		5		
Late_blight		1	91	1	2	2			
Leaf_Mold				89					
Septoria_leaf_spot		2	2	3	91		3		
Two-spotted_spider_mite				2		91	1		
Target_Spot	2	1			1	1	84		
Yellow_Leaf_Curl_Virus								88	
healthy						1	2		95

(e) SqueezeNet

	Bacterial_spot	Early_blight	Late_blight	Leaf_Mold	Septoria_leaf_spot	Two-spotted_spider_mite	Target_Spot	Yellow_Leaf_Curl_Virus	healthy
Bacterial_spot	94	1							
Early_blight		88			1				
Late_blight		5	95				1		
Leaf_Mold				95		2			
Septoria_leaf_spot					94		1		
Two-spotted_spider_mite						91			
Target_Spot	1	1				1	92		
Yellow_Leaf_Curl_Virus								95	
healthy						1	1		95

(f) EfficientNetB0

Fig. 7. Confusion matrices representing predicted classes against true classes of GoogleNet (a), MobileNetV2 (b), NasNetMobile (c), ResNet-18 (d), SqueezeNet (e), and EfficientNetB0 (f).

target spot disease and mixed its features with two spotted disease and healthy class. The remaining models were able to effectively learn the features of target spot disease and distinguish it from other classes. For example, GoogleNet (Fig. 7a) and MobileNetV2 (Fig. 7b) accurately predicted the diseases in most cases. However, GoogleNet misclassified 7 images of target spot diseases into other categories, especially two spotted spider mite (3 instances), early blight (2 instances), bacterial spot (1 instance), and septoria leaf spot (1 instance). On the other hand,

Table 3
Classification time results between the CNN models and the 4 configurations utilized in our study.

Model/Images		NVIDIA Jetson AGX ORIN (in seconds)	NVIDIA Jetson NANO ORIN (in seconds)	RPI 4 (in seconds)	RPI 4 with Google Coral (in seconds)
ResNet-18	1 image	0.000489	0.000895	0.00291	0.00168
	855 images	0.418	0.766	2.486	1.436
EffcientNetB0	1 image	0.000546	0.000821	0.00252	0.00147
	855 images	0.467	0.702	2.208	1.259
NasNetMobile	1 image	0.000681	0.000997	0.00282	0.00171
	855 images	0.582	0.853	2.408	1.46
SqueezeNet	1 image	0.000351	0.000522	0.00153	0.00105
	855 images	0.300	0.446	1.311	0.898
MobileNetv2	1 image	0.000437	0.000630	0.00184	0.00116
	855 images	0.374	0.539	1.576	0.9918
GoogleNet	1 image	0.000501	0.000901	0.00289	0.00163
	855 images	0.428	0.770	2.474	1.39

because SqueezeNet (Fig. 7e) has the least number of parameters, it was effectively able to distinguish the target spot disease. However, its misclassification rate was distributed across various diseases, resulting in lower overall accuracy compared to GoogleNet, MobileNetV2, ResNet-18, and EfficientNetB0.

3.2. *Performance of edge-AI systems*

We measured the classification time for a single randomly selected sample image from the total classes for all configurations. Additionally, we classified 95 images per class, resulting in a total of 855 images across all classes. It is worth mentioning that, to be consistent, the same images were used for testing Edge devices as were used for the model performances (discussed in Section 3.1).

The results presented in Table 3 highlight the superior performance of the NVIDIA Jetson ORIN AGX running SqueezeNet. This configuration achieved a remarkable classification time of 3.5 milliseconds for a single image and completed the classification of 855 images in just 0.3 s. Notably, SqueezeNet's inference speed surpassed that of other classifiers across all platforms assessed.

Furthermore, we assessed the GPU and CPU utilization (as shown in Fig. 8a), and CPU temperature (as shown in Fig. 8b) for models operating on power-efficient devices. To offer more comprehensive insights, we transformed still-images into a 6-minute video. These images were randomly selected from all classes, with each frame displayed for 2 s before transitioning to the next. The power consumption for the Jetson Orin AGX was 45 W, while for the Jetson Orin Nano 15 W. Similarly, the RPI 4 operates in a power mode of 15 W. Figs. 8a and 8b present the inference performance results for various performance indicators, including average CPU utilization, average GPU utilization, and average CPU temperature ratings in Celsius. These measurements were conducted on NVIDIA Jetson Orin Nano, NVIDIA Jetson Orin AGX, and RPI 4 running Google Coral, respectively.

In Table 3, the inference performance for NVIDIA Jetson Orin Nano across various models is depicted. Notably, the SqueezeNet model demonstrates higher performance on Jetson Nano, achieving an FPS of 1915, outperforming other models. While there are minimal differences in performance metrics such as average CPU utilization, average GPU utilization, and average CPU temperature ratings among models, it is observed that the average CPU and GPU utilization remain low for Jetson Nano. Additionally, the average CPU temperature remains within acceptable limits, indicating no thermal constraints. For the NVIDIA Jetson Orin AGX across different models, the SqueezeNet model showcases the highest FPS of 2849, followed by MobileNetV2 with 2288 FPS, exhibiting excellent performance. Similar to the Orin Nano results, there are negligible differences in performance metrics among models, and the average temperature remains within acceptable ranges. The FPS for SqueezeNet running on RPI 4 Model B with Google Coral is reported at 952, surpassing other models. However, the average CPU utilization for all models ranges from 89 % to 92 %, nearing its limits, indicating the computational intensity of the models for RPI 4. GPU utilization remains consistent across models, indicating a high demand for GPU computation in object classification. Despite this, CPU temperatures remain below thermal limits, attributed to the presence of a PWM fan, ensuring optimal operation.

Fig. 9 displays the average device power consumption observed across various models during algorithm execution. The theoretical absorbed power differs across different operational modes from the actual dissipated power during execution. Results indicate that the NVIDIA Jetson Orin AGX demonstrates higher power demands, which aligns with expectations given its higher classification speed/FPS. Across different devices/platforms, there's a slight variance in power consumption among models, with measured values remaining fairly consistent for each model.

Furthermore, a comparison of embedded platforms in terms of cost/FPS ratio is presented in Table 4, considering SqueezeNet as the benchmark model. RPI 4 with Google Coral emerges as the most balanced choice in terms of cost and performance despite additional costs associated with the Google Coral hardware accelerator. Indeed, Google Coral is a hardware USB accelerator and cannot be used as a stand-alone device for real time object detection. This indicates that an additional embedded device [50] must be purchased for the Google Coral, such as RPI, thus increasing its final cost. While NVIDIA Jetson Orin AGX offers superior quality, its higher cost may not be suitable for low-cost solutions. Conversely, NVIDIA Jetson Orin Nano provides a more favorable cost/FPS ratio, positioning it as a competitive option.

Analysis of the performance results presented in Tables 3 and 4 reveals notable differences among the models tested, with NVIDIA Jetson AGX Orin exhibiting superior performance compared to NVIDIA Jetson Orin Nano and RPI 4 with Google Coral. This performance disparity can be attributed to the higher quality board of the Jetson AGX Orin. Considering various evaluation metrics such as resource usage (CPU and GPU utilization), CPU temperature, FPS, and average power consumption, the suitability of deep learning frameworks for the Jetson AGX Orin can be assessed as follows: If prioritizing classification speed while minimizing resource usage relative to cost, the NVIDIA Jetson AGX Orin emerges as the preferred choice. Moreover, for large-scale farming operations where the cost of Edge devices is feasible, the NVIDIA Jetson Orin AGX proves to be a suitable option. On the other hand, the NVIDIA Jetson Orin Nano offers commendable performance in resource usage and classification speed, coupled with reasonable average power consumption at a more affordable cost. Conversely, the RPI 4 with Google Coral exhibits slightly lower performance but remains adequate for classification tasks with acceptable speed and accuracy, albeit with higher resource usage and potential reliability concerns over prolonged use. Thus, for short-term classification needs and small-scale farming operations where cost is a critical factor, the RPI 4 with Google Coral could be considered. In summary, the choice of the most suitable platform depends on specific requirements such as classification speed, resource usage, average power consumption, cost, and the scale of the farming operation. Each platform offers distinct advantages and tradeoffs, allowing for tailored solutions to meet varying application needs.

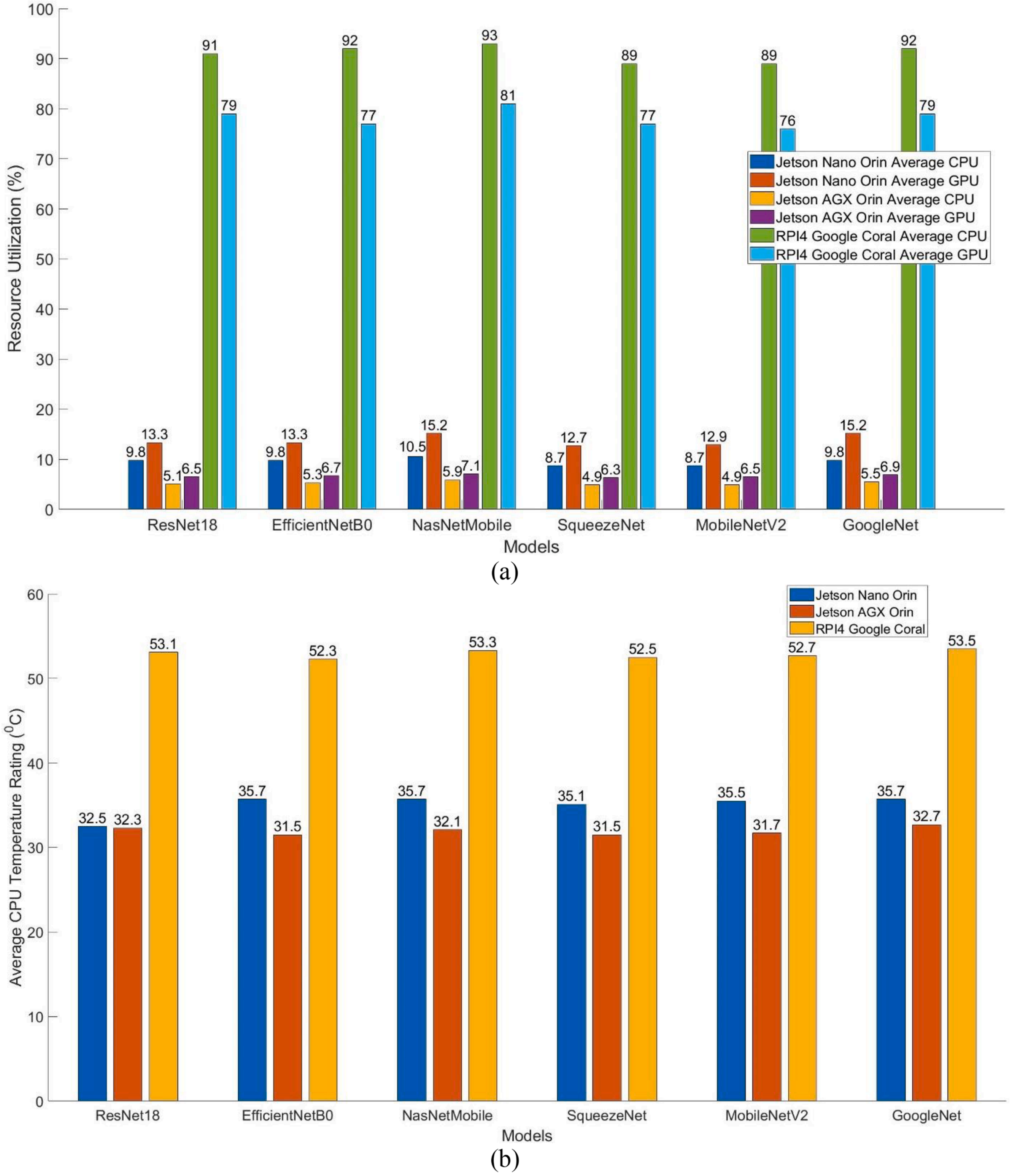


Fig. 8. Resource utilization (both GPU and CPU) (a) and average CPU temperature rating (b) for all CNN networks running on Edge devices (NVIDIA Jetson Nano Orin, NVIDIA AGX Orin, and RPI 4 with and without Google Coral).

Overall, this study used the open-sourced dataset, which was collected with constant background conditions encompassing different tomato leaf diseases. However, for accurate disease detection for real-field disease monitoring, a diverse dataset encompassing varying background and illumination conditions needs to be collected, and deep learning algorithms need to be trained for that dataset. As the results indicated that edge device performance has a positive correlation with the number of parameters of the trained networks (a lower number of parameters results in high FPS with lower CPU and GPU usage); therefore, designing custom lightweight networks tailored to the tomato disease detection will results in a less complex network with less number of parameters as compared to using pre-trained network hence

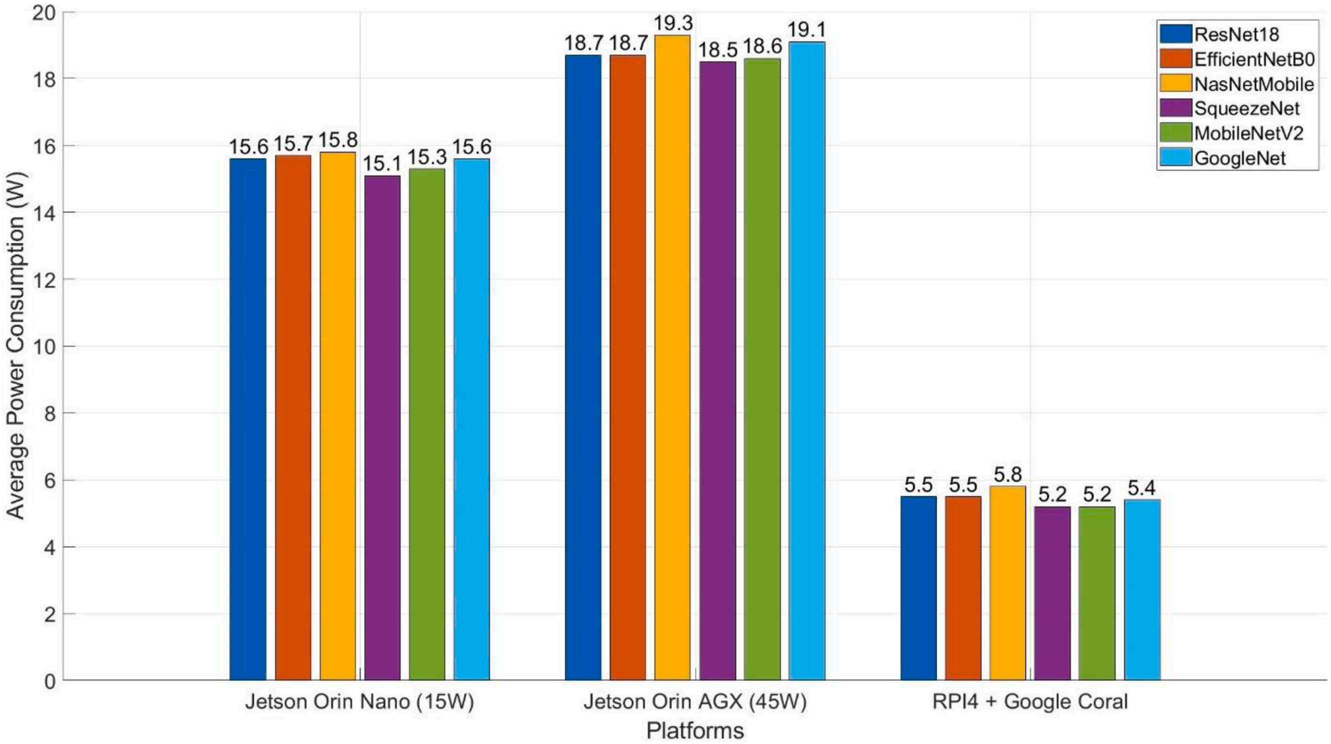


Fig. 9. Average Edge device power consumption observed across various CNN models during algorithm execution.

Table 4
Cost/performance analysis of the different embedded platforms running SqueezeNet classification model.

Device	Cost ($) Starting at	FPS	Cost/FPS
NVIDIA Jetson AGX Orin	1999	2849	0.7
NVIDIA Jetson Nano Orin	499	1915	0.26
RPI 4 Model B (8GB) with Coral	135	952	0.14

increasing the performance and reliability of the Edge-AI solutions. Customized lightweight models particularly optimized for the limited computational resources of edge devices will ensure faster inference times, lower power consumption, and enhanced reliability. Additionally, future studies will focus on designing the diseased region detection and/or segmentation networks which could pave the way for precise and targeted spraying applications for disease management.

Edge devices typically have lower processing power (CPUs/GPUs) compared to cloud servers. While there are exceptions, such as the powerful but expensive NVIDIA Jetson Orin AGX, most edge devices have limited computational resources. Complex CNN models demand intensive computations for inference, which can result in slow inference times and limited model complexity on these devices. Running large CNN models on edge devices might be impractical, leading to trade-offs between accuracy and efficiency. Memory constraints and energy efficiency are additional limitations of edge devices for deploying CNN models. Large CNN models with numerous parameters require significant memory for execution. Moreover, power consumption is a critical concern, as running computationally intensive CNN models can quickly drain the batteries of most edge devices, which are typically battery powered. Scalability could be another challenge because of the difficulty in managing and maintaining a large number of edge devices running CNN models. These limitations necessitate careful consideration when deploying CNNs on edge devices, particularly for tasks like early tomato disease detection. High-performance deep learning CNNs require significant computing resources, such as devices equipped with GPUs, to achieve high accuracy. However, their substantial processing power demands make them unsuitable for embedded systems with limited computational capabilities. This poses a challenge for deploying CNNs on edge devices typically used in precision agriculture, which often lack the necessary processing power for these complex models. In contrast, lightweight CNNs are specifically designed for efficiency, making them ideal for these devices. Our study evaluated four different hardware platforms, each varying in computational resources and cost, to assess the performance of lightweight CNN models on these platforms. The results highlight the performance accuracy of each hardware platform with the lightweight models, providing valuable insights for farmers of all scales—small, medium, and large—in choosing the right platform based on performance, power consumption, and cost considerations. A crucial takeaway from our study is the importance of data privacy and security. Since edge devices collect and process data locally, addressing these concerns is essential. Lightweight CNNs help mitigate data transmission over networks, enhancing data privacy and security. Additionally, potential applicability of our findings to various agricultural use cases, such as crop monitoring, pest detection, yield prediction, and livestock monitoring. Our study underscores the critical role of identifying the optimal hardware and software configurations for these applications. Edge computing and IoT technologies are well-suited for AI-based applications and offer substantial cost benefits. However, despite the readiness of Edge-AI technology, deploying innovative applications remains a challenging task. Overall, our study demonstrates that while Edge-AI technology is mature enough for practical use, careful consideration of hardware and software configurations is necessary to overcome deployment challenges and fully leverage its potential in agricultural applications.

4. Conclusion

The primary goal of this study was to evaluate the performance of standalone Edge-AI solutions for tomato leaf disease detection. To achieve this goal, lightweight deep learning networks to detect and differentiate tomato leaf diseases were deployed on low-cost and low-power consumption Edge devices to investigate their performance capabilities as standalone Edge-AI solutions. Lightweight CNN based deep learning networks achieved the accuracies of up to 98.25 % in detecting tomato diseases. RPI 4 with AI accelerator (Google Coral) achieved the best cost/FPS performance of 0.14 compared to other Edge devices NVIDIA Jetson AGX Orin and NVIDIA Jetson Nano Orin with cost/FPS of 0.7 and 0.26, respectively. While NVIDIA Jetson Orin AGX offers superior quality, its higher cost may not be suitable for low-cost solutions. Conversely, NVIDIA Jetson Orin Nano provides a more favorable cost/FPS ratio, resource usage (CPU and GPU utilization), CPU temperature, and average power consumption, and the suitability of deep learning frameworks, positioning it as a competitive option for potential reliability concerns over prolonged use as standalone Edge-AI solution for agricultural applications. Our study demonstrates that while Edge-AI technology is mature enough for practical use, careful consideration of hardware and software configurations is necessary to overcome deployment challenges and fully leverage its potential in agricultural applications. Future studies will focus on designing custom lightweight CNN algorithms by collecting real-field disease datasets and evaluating the performance of Edge-AI in real-field conditions. Customized lightweight models particularly optimized for the limited computational resources of edge devices will ensure faster inference times, lower power consumption, and enhanced reliability. In addition, we will also conduct extensive field trials to validate the effectiveness of these Edge-AI solutions in diverse agricultural settings. Lastly, we will also explore user-friendly interfaces that will make these technologies accessible to farmers of all scales.

Ethics statement

Not applicable: This manuscript does not include human or animal research

CRediT authorship contribution statement

Yaqoob Majeed: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Mike O. Ojo:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Azlan Zahid:** Writing – review & editing, Supervision, Resources, Project administration, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgement

This research was funded by the Texas Department of Agriculture (Specialty Crop Block Grant Agreement # GSC2022105). This research is partially supported by the United States Department of Agriculture (USDA)'s National Institute of Food and Agriculture (NIFA) Federal Appropriations under TEX09954 and Accession No. 7002248. This publication is also supported by Texas A&M AgriLife Research, Vegetable and Fruit Improvement Center (VFIC).

References

[1] FAO (Food and Agriculture Organization of the United Nations), FAOSTAT Database, 2022. https://www.fao.org/faostat/en/#home.
[2] B. Jin, X. Xu, Forecasting wholesale prices of yellow corn through the Gaussian process regression, Neural Comput. Applicat. 36 (15) (2024) 8693–8710.
[3] X. Xu, Y. Zhang, Corn cash price forecasting with neural networks, Comput. Electron. Agric. 184 (2021) 106120.
[4] A. Abade, P.A. Ferreira, F de Barros Vidal, Plant diseases recognition on images using convolutional neural networks: a systematic review, Comput. Electron. Agric. 185 (2021) 106125.
[5] B. Jin, X. Xu, Price forecasting through neural networks for crude oil, heating oil, and natural gas, Measurement 1 (1) (2024) 100001.
[6] H.F. Pardede, E. Suryawati, D. Krisnandi, R.S. Yuwana, V. Zilvan, Machine learning based plant diseases detection: a review, in: 2020 International conference on radar, antenna, microwave, electronics, and telecommunications (ICRAMET), IEEE, 2020, pp. 212–217.
[7] H.D. Gadade, D.K. Kirange, Machine learning based identification of tomato leaf diseases at various stages of development, in: 2021 5th international conference on computing methodologies and communication (ICCMC), IEEE, 2021, pp. 814–819.
[8] R.R. Kovvuri, A. Kaushik, S. Yadav, Disruptive technologies for smart farming in developing countries: tomato leaf disease recognition systems based on machine learning, Electr. J. Info. Syst. Develop. Count. 89 (6) (2023) e12276.
[9] U. Mokhtar, M.A. Ali, A.E. Hassenian, H. Hefny, Tomato leaves diseases detection approach based on support vector machines, in: 2015 11th International computer engineering conference (ICENCO), IEEE, 2015, pp. 246–250.
[10] M.S.T. Anubhove, N. Ashrafi, A.M. Saleque, M. Akter, S.U. Saif, Machine learning algorithm based disease detection in tomato with automated image telemetry for vertical farming, in: 2020 International conference on computational performance evaluation (ComPE), IEEE, 2020, pp. 250–254.
[11] S.S. Harakannanavar, J.M. Rudagi, V.I. Puranikmath, A. Siddiqua, R. Pramodhini, Plant leaf disease detection using computer vision and machine learning algorithms, Global Transit. Proceed. 3 (1) (2022) 305–310.
[12] C. Nyasulu, A. Diattara, A. Traore, C. Ba, P.M. Diedhiou, Y. Sy, D.H. Peluffo-Ordóñez, A comparative study of Machine Learning-based classification of Tomato fungal diseases: application of GLCM texture features, Heliyon. 9 (11) (2023).
[13] S.A. Kumar, S. Sasikala, Disease detection in tomato leaves using machine learning and statistical feature fusion, in: 2021 International conference on advancements in electrical, electronics, communication, computing and automation (ICAECA), IEEE, 2021, pp. 1–6.
[14] S.M. Kiran, D.N. Chandrappa, Plant leaf disease detection using efficient image processing and machine learning algorithms, J. Robotics Control (JRC) 4 (6) (2023) 840–848.
[15] M. Govardhan, M.B. Veena, Diagnosis of tomato plant diseases using random forest, in: 2019 Global conference for advancement in technology (GCAT), IEEE, 2019, pp. 1–5.
[16] M. Chen, Z. Chen, L. Luo, Y. Tang, J. Cheng, H. Wei, J. Wang, Dynamic visual servo control methods for continuous operation of a fruit harvesting robot working throughout an orchard, Comput. Electron. Agric. 219 (2024) 108774.
[17] R. Thangaraj, P. Pandiyan, S. Anandamurugan, S. Rajendar, A deep convolution neural network model based on feature concatenation approach for classification of tomato leaf disease, Multimed. Tools. Appl. (2023) 1–25.
[18] R. Bora, D. Parasar, S. Charhate, A detection of tomato plant diseases using deep learning MNDLNN classifier, Signal. Image Video Process. (2023) 1–9.
[19] P. Kaur, S. Harnal, V. Gautam, M.P. Singh, S.P. Singh, A novel transfer deep learning method for detection and classification of plant leaf disease, J Ambient Intell. Humaniz. Comput. 14 (9) (2023) 12407–12424.
[20] M. Nawaz, T. Nazir, A. Javed, M. Masood, J. Rashid, J. Kim, A. Hussain, A robust deep learning approach for tomato plant leaf disease localization and classification, Sci. Rep. 12 (1) (2022) 18568.
[21] A. Fuentes, S. Yoon, S.C. Kim, D.S. Park, A robust deep-learning-based detector for real-time tomato plant diseases and pests recognition, Sensors 17 (9) (2017) 2022.
[22] Y. Wang, Y. Chen, D. Wang, Convolution network enlightened transformer for regional crop disease classification, Electronics. 11 (19) (2022) 3174.
[23] K.P. Ferentinos, Deep learning models for plant disease detection and diagnosis, Comput. Electron. Agric. 145 (2018) 311–318.
[24] K. Roy, S.S. Chaudhuri, J. Frnda, S. Bandopadhyay, I.J. Ray, S. Banerjee, J. Nedoma, Detection of tomato leaf diseases for agro-based industries using novel PCA DeepNet, IEEe Access. 11 (2023) 14983–15001.
[25] X. Chen, G. Zhou, A. Chen, J. Yi, W. Zhang, Y. Hu, Identification of tomato leaf diseases based on combination of ABCK-BWTR and B-ARNet, Comput. Electron. Agric. 178 (2020) 105730.
[26] F. Meng, J. Li, Y. Zhang, S. Qi, Y. Tang, Transforming unmanned pineapple picking with spatio-temporal convolutional neural networks, Comput. Electron. Agric. 214 (2023) 108298.
[27] L. Ye, F. Wu, X. Zou, J. Li, Path planning for mobile robots in unstructured orchard environments: an improved kinematically constrained bi-directional RRT approach, Comput. Electron. Agric. 215 (2023) 108453.
[28] P.P. Ray, D. Dash, D. De, Edge computing for Internet of Things: a survey, e-healthcare case study and future direction, J. Network Comput. Applicat. 140 (2019) 1–22.
[29] E. Badidi, K. Moumane, F. El Ghazi, Opportunities, applications, and challenges of edge-AI enabled video analytics in smart cities: a systematic review, IEEe Access. (2023).
[30] A. Abraham, S. Dash, J.J. Rodrigues, B. Acharya, S.K. Pani (Eds.), AI, edge and iot-based smart agriculture, Academic Press, 2021.

[31] R. Sanchez-Iborra, A.F. Skarmeta, Tinyml-enabled frugal smart objects: challenges and opportunities, IEEE Circuits Syst. Magaz. 20 (3) (2020) 4–18.

[32] P.T. Anh, H.T.M. Duc, A benchmark of deep learning models for multi-leaf diseases for edge devices, in: 2021 International conference on advanced technologies for communications (ATC), IEEE, 2021, pp. 318–323.

[33] V. Gonzalez-Huitron, J.A. León-Borges, A.E. Rodriguez-Mata, L.E. Amabilis-Sosa, B. Ramírez-Pereda, H. Rodriguez, Disease detection in tomato leafs via CNN with lightweight architectures implemented in Raspberry Pi 4, Comput. Electron. Agric. 181 (2021) 105951.

[34] H. Tarek, H. Aly, S. Eisa, M Abul-Soud, Optimized deep learning algorithms for tomato leaf disease detection with hardware deployment, Electronics. 11 (1) (2022) 140.

[35] R. Moreira, L.F.R. Moreira, P.L.A. Munhoz, E.A. Lopes, R.A.A. Ruas, AgroLens: a low-cost and green-friendly Smart Farm Architecture to support real-time leaf disease diagnostics, Internet Things 19 (2022) 100570.

[36] Y. Zhang, M. Chen, An IoT-enabled energy-efficient approach for the detection of leaf curl disease in tomato crops, Wireless Networks 29 (1) (2023) 321–329.

[37] A. Nag, P.R. Chanda, S. Nandi, Mobile app-based tomato disease identification with fine-tuned convolutional neural networks, Comput. Electric. Eng. 112 (2023) 108995.

[38] D. Hughes, M. Salathé, et al., An open access repository of images on plant health to enable the development of mobile disease diagnostics, arXiv preprint, 2015. arXiv:1511.08060.

[39] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 770–778.

[40] M. Tan, Q. Le, Efficientnet: rethinking model scaling for convolutional neural networks, in: International conference on machine learning, PMLR, 2019, pp. 6105–6114.

[41] B. Zoph, V. Vasudevan, J. Shlens, Q.V. Le, Learning transferable architectures for scalable image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 8697–8710.

[42] F.N. Iandola, S. Han, M.W. Moskewicz, K. Ashraf, W.J. Dally, K. Keutzer, SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size, arXiv preprint, 2016 *arXiv:1602.07360*.

[43] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.C. Chen, Mobilenetv2: inverted residuals and linear bottlenecks, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 4510–4520.

[44] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, A. Rabinovich, Going deeper with convolutions, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2015, pp. 1–9.

[45] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, L. Fei-Fei, Imagenet large scale visual recognition challenge, Int. J. Comput. Vis. 115 (2015) 211–252.

[46] J. Winzig, J.C.A. Almanza, M.G. Mendoza, T. Schumann, Edge AI - use case on google coral dev board mini, in: 2022 IET International conference on engineering technologies and applications (IET-ICETA), 2022, pp. 1–2, https://doi.org/10.1109/IET-ICETA56553.2022.9971614.

[47] N. Gabdullin, A. Raskovalov, Google Coral-based edge computing person reidentification using human parsing combined with analytical method, Internet Things 22 (2023) 100701, https://doi.org/10.1016/j.iot.2023.100701.

[48] S. Mittal, A Survey on optimized implementation of deep learning models on the NVIDIA Jetson platform, J. Syst. Archit. 97 (2019) 428–442, https://doi.org/10.1016/j.sysarc.2019.01.011.

[49] S. Mittal, J.S. Vetter, A Survey of CPU-GPU heterogeneous computing techniques, ACM. Comput. Surv. 47 (4) (2015) 1–35, https://doi.org/10.1145/2788396.

[50] M.O. Ojo, S. Giordano, G. Procissi, I.N. Seitanidis, A review of low-end, middle-end, and high-end Iot devices, IEEe Access. 6 (2018) 70528–70554, https://doi.org/10.1109/ACCESS.2018.2879615.